Assignment-8

* Problem Statement: Write a C program to evaluate postfix expression using Stack.

* Objective: i) To study stack & its operations.

ii) To study the importance of expression evaluation.

* Test Conditions:

Test Case	Input	Expected O/P
1.	23+	5
2.	23*	6
3.	23+4*	20
4.	52-3*	6

* Pseudo Code:

```
Fuction evalPostfix (exp)
    Initialize an empty stack

  For each token in exp from left to right

     If token is an operand then
        Push → Token

     Else if token = operator then

        Pop top 2 operands from stack

        result = Calculate(operand 1 ,token, operand2)

        result -> Push

     End If
  End For

   Return Pop from Stack
End Function

   Function calculate (Op1, operator, op2)

      Switch ~~case~~: (operator):
           Case '+': return op1 + op2
           Case '-': return op1 - Op2
           case '*': return op1 * op2
           Case '/': return op1/op2
      End Switch
End Function.
```

- Time Complexity:

Operation	Time Complexity
PUSH	O(1)
POP	O(1)
Postfix Evaluation	O(n)

* Conclusion:

Thus, implemented stack operations to evaluate the value of an expression.

* FAQs:

Q1. Explain how postfix / prefix expression are evaluated.

A- • Postfix: Scan from left to right push operands into the stack. When an operator is found, pop top two operands, perform the operation by putting it like - Op1 operator Op2 & push the result.

• Prefix: Scan from right to left. Push the operands in the stack. When an operator is found, pop top two operands & place operator in between to perform operation. Store the result back in Stack using push.

Q2. What is ISP & ICP?

A- • ISP (In Stack Priority): Priority of operator when it is already present inside stack.

• ICP (Incoming Priority): Priority of an operator when it is being inserted into the Stack.

- Mainly used during infix to post/prefix conversion.

Q3. Give various applications of Stack.

A- i) Expression Conversion
ii) Function call & Recursion
iii) Paranthesis Matching
iv) Undo/Redo Operations
v) Backtracking
vi) Browser History
vii) Depth Fist Search Algorithm (DSF).

PRACTICAL 8

EXPRESSION EVALUATION USING STACK

CSF20030 – Data Structures Laboratory

Experiment 08: Evaluate a Postfix Expression

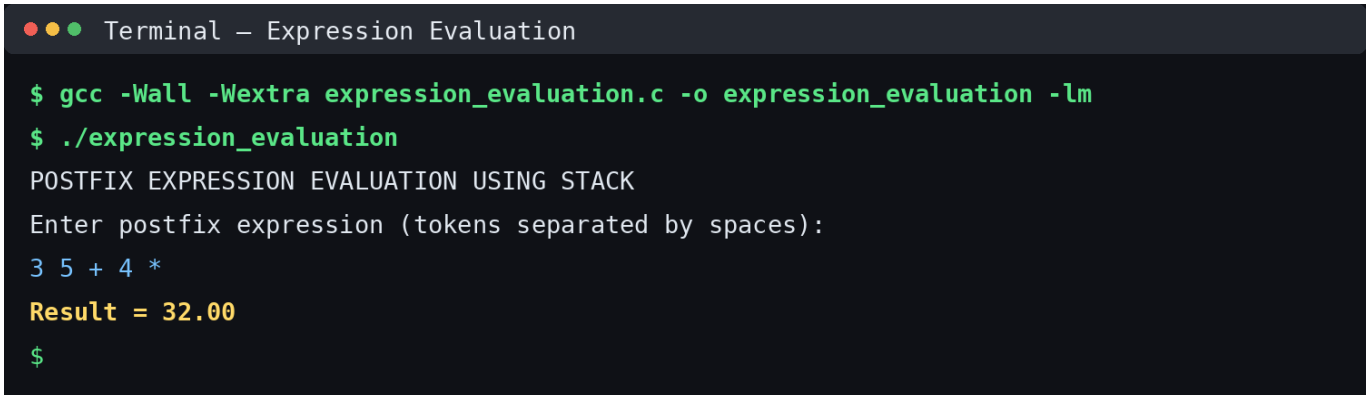
Aim: Write a C program to evaluate a postfix expression using a stack.

C Program

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#define MAX 200
double stack[MAX];
int top = -1;
void push(double value) {
    if (top == MAX - 1) {
        printf("Error: Stack overflow.\n");
        exit(EXIT_FAILURE);
    }
    stack[++top] = value;
}
double pop(void) {
    if (top == -1) {
        printf("Error: Invalid postfix expression (stack underflow).\n");
        exit(EXIT_FAILURE);
    }
    return stack[top--];
}
/* Evaluate postfix expression with space-separated tokens.
   Supports multi-digit numbers and + - * / % ^ operators. */
double evaluatePostfix(char expression[]) {
    char *token = strtok(expression, " \t\n");
    while (token != NULL) {
        char *endptr;
        double value = strtod(token, &endptr);
        if (endptr != token && *endptr == '\0') {
            push(value);
        } else if (strlen(token) == 1 &&
                   strchr("+-*/%^", token[0]) != NULL) {
            double b = pop();
            double a = pop();
            switch (token[0]) {
                case '+': push(a + b); break;
                case '-': push(a - b); break;
                case '*': push(a * b); break;
                case '/':
                    if (b == 0) {
                        printf("Error: Division by zero.\n");
                        exit(EXIT_FAILURE);
                    }
                    push(a / b);
                    break;
                case '%':
                    if (b == 0) {
                        printf("Error: Modulo by zero.\n");
                        exit(EXIT_FAILURE);
                    }
                    push(fmod(a, b));
                    break;
                case '^': push(pow(a, b)); break;
            }
        } else {
            printf("Error: Invalid token '%s'.\n", token);
            exit(EXIT_FAILURE);
        }
        token = strtok(NULL, " \t\n");
    }
    if (top != 0) {
        printf("Error: Invalid postfix expression.\n");
        exit(EXIT_FAILURE);
    }
    return pop();
}
int main(void) {
    char expression[500];
    printf("POSTFIX EXPRESSION EVALUATION USING STACK\n");
    printf("Enter postfix expression (tokens separated by spaces):\n");
    if (fgets(expression, sizeof(expression), stdin) == NULL) {
        printf("Input error.\n");
        return 1;
    }
```

```
    top = -1;
    double result = evaluatePostfix(expression);
    printf("Result = %.2f\n", result);
    return 0;
}
```

Sample Input and Output Screenshot



Input: 3 5 + 4 *
Output: Result = 32.00
The expression is evaluated as (3 + 5) × 4 = 32.